SESSION 07:

Class Diagram

Module : Phân tích và thiết kế hệ thống

Version: 1.0

RIKKEI Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NỘI DUNG

1. Tổng quan về Class trong UML
2. Bổ từ truy cập
3. Các mối quan hệ
4. Bội số trong Class Diagram
5. Từ Use Case —> Class Diagram
6. Hướng dẫn: Trích xuất và vẽ Class Diagram

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Lớp (Class) là gì?

Một Class là một **khuôn mẫu (template/blueprint)** để tạo ra các đối tượng có cùng thuộc tính (đặc điểm) và phương thức (hành vi).

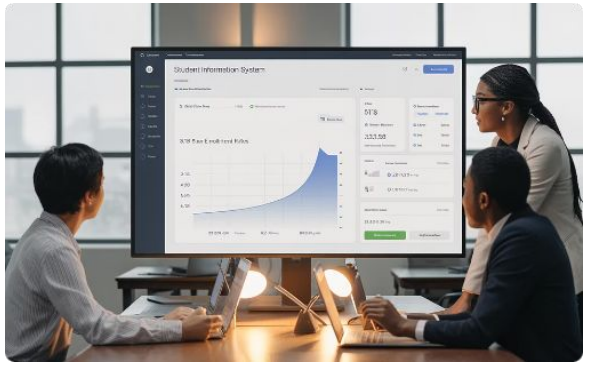
Class là cái chung, Object là cái riêng



Khuôn làm bánh

Class: Khuôn làm bánh hình tròn

Object: Những chiếc bánh cụ thể được tạo ra từ cái khuôn đó



Hệ thống quản lý

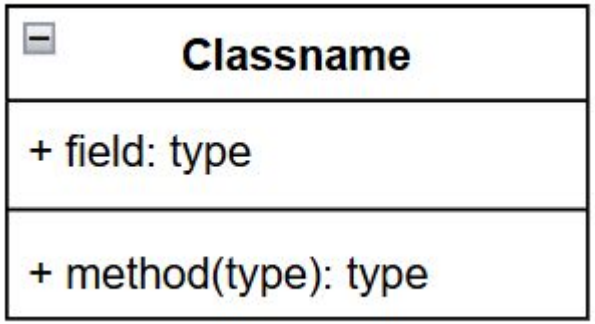
Class: Student

Object: Sinh viên cụ thể "Nguyễn Văn An", "Trần Thị Bình"

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Ký hiệu của class



Hình chữ nhật chia làm 3 phần

01

Tên Lớp (Class Name)

Luôn là **DANH TỪ**, viết hoa chữ cái đầu

Ví dụ: SinhVien, MonHoc, GiangVien

02

Thuộc tính (Attributes)

Các thông tin, đặc điểm mà Class đó có

Ví dụ: maSV, hoTen, ngaySinh, diemTB

03

Phương thức (Operations/Methods)

Các hành động, hành vi mà Class có thể thực hiện

Ví dụ: tinhDiemTrungBinh(), dangKyMonHoc()

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Class diagram là gì?

Định nghĩa chính thức

Là một biểu đồ cấu trúc tĩnh, mô tả các **lớp** trong hệ thống, các **thuộc tính** và **phương thức** của chúng, và quan trọng nhất là các **mối quan hệ** giữa các lớp đó.

Không chỉ là tập hợp đơn thuần

Nó không chỉ là một tập hợp các lớp riêng lẻ, mà là một **sơ đồ mạng lưới** cho thấy cách các khối thông tin kết nối và tương tác với nhau.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Vai trò của class diagram



Cầu nối Phân tích - Lập trình

Chuyển hóa các yêu cầu nghiệp vụ thành cấu trúc logic mà lập trình viên có thể dựa vào để viết code hiệu quả.



Nền tảng thiết kế Database

Từ Class Diagram, chúng ta dễ dàng suy ra các bảng cần có trong databse. Ví dụ: Class SinhVien → Bảng SINHVIEN.



Công cụ thảo luận thiết kế

Cả đội ngũ (phân tích, lập trình, kiểm thử) có thể nhìn vào cùng một bản vẽ để thảo luận và cải tiến thiết kế trước khi code.



Tăng khả năng bảo trì

Thiết kế tốt dựa trên Class Diagram giúp hệ thống dễ dàng sửa đổi, nâng cấp và tái sử dụng các thành phần trong tương lai.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Thuộc tính (Atributes)

Class BIẾT những gì?

Thuộc tính là các dữ liệu, thông tin, đặc điểm mà một đối tượng của class đó sẽ lưu trữ. Đây chính là "bộ nhớ" của đối tượng, nơi chứa đựng tất cả những gì nó cần "nhớ" về bản thân.

Cú pháp chuẩn UML

```
visibility name: type [= defaultValue]
```

Các thành phần

visibility: Mức độ truy cập (+, -, #, ~)

name: Tên thuộc tính, viết camelCase (VD: fullName, averageScore)

type: Kiểu dữ liệu (String, int, double, Date)

defaultValue: Giá trị khởi tạo (tùy chọn)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Phương thức (Methods)

Class LÀM được gì?

Phương thức là các hành động, hành vi, thao tác mà một đối tượng của class có thể thực hiện. Nếu thuộc tính là "bộ nhớ", thì phương thức chính là "kỹ năng" của đối tượng.

01

Xác định hành động

Đối tượng này có thể làm gì?

02

Thiết kế tham số

Cần thông tin gì để thực hiện?

03

Xác định kết quả trả về

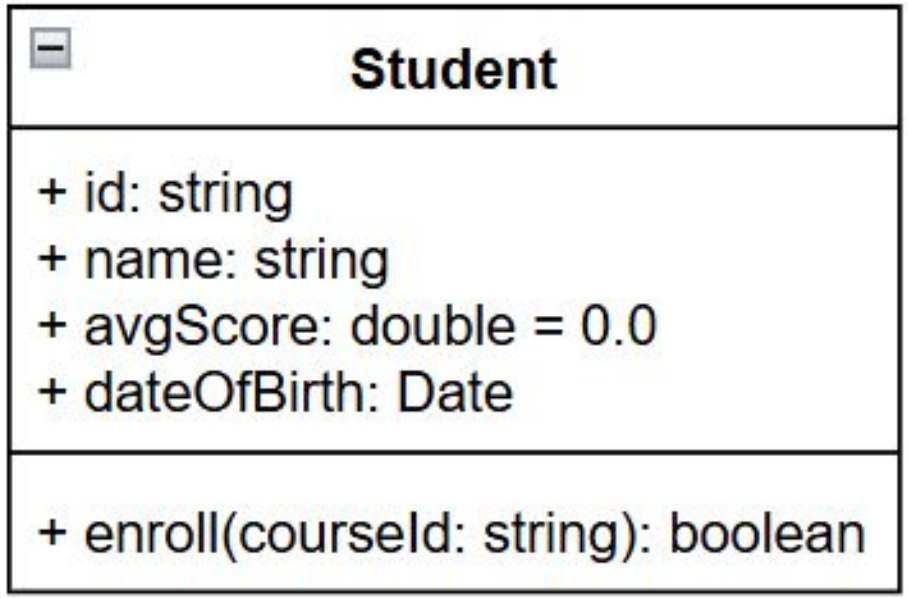
Sau khi thực hiện, có gì được trả về?

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Rikkei Academy
where the dream begins

Class hoàn chỉnh



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Thành phần static

"CỦA RIÊNG" và "CỦA CHUNG"



Thành phần Instance (Mặc định)

Mỗi **đối tượng** sẽ có một bản sao riêng của thuộc tính/phương thức này. Giống như mỗi người có tên riêng, tuổi riêng vậy.



Thành phần Static (Gạch chân)

Thuộc tính/phương thức này là **CỦA CHUNG**, được chia sẻ cho TẤT CẢ các đối tượng. Nó thuộc về Class, không thuộc riêng ai.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Tổng quan về Class trong UML

Lớp và phương thức trừu tượng (Abstract)

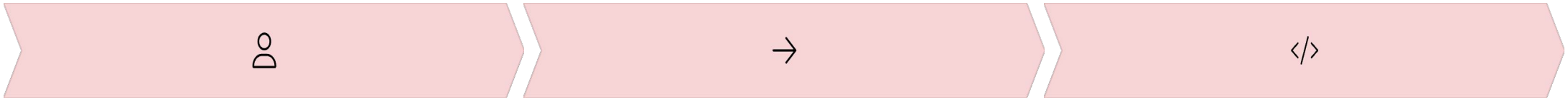
Phương thức trừu tượng

Là phương thức chỉ có **khai báo**, không có **phần thân**. Nó chỉ nói "phải có hành động này" chứ không nói "làm như thế nào".

Lớp trừu tượng

Là lớp có chứa ít nhất một phương thức trừu tượng. **Không thể tạo đối tượng** trực tiếp từ lớp này.

Ký hiệu UML: Viết *in nghiêng (italic)*



Abstract Class

Định nghĩa khuôn mẫu chung

Kế thừa

Class con phải implement

Concrete Class

Cài đặt cụ thể các method

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Bổ từ truy cập (Access modifiers)

Mục Tiêu Bài Học

Hiểu về Tính Đóng Gói

Nắm vững khái niệm Encapsulation và tầm quan trọng của việc bảo vệ dữ liệu trong lập trình hướng đối tượng

Thành Thạo Access Modifiers

Hiểu rõ 4 loại Access Modifiers trong UML: public (+), private (-), protected (#), package (~) và cách sử dụng phù hợp

Áp Dụng Nguyên Tắc Thiết Kế

Nắm vững nguyên tắc "Private Data, Public Methods" - nền tảng của thiết kế class tốt

Làm Quen Getter & Setter

Hiểu cách sử dụng Getter và Setter để tương tác an toàn với dữ liệu private

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Bổ từ truy cập (Access modifiers)

Tại Sao Cần "Cửa Và Khóa"?

Tình huống nguy hiểm:

Nếu mọi thuộc tính của Class SinhVien đều public, lập trình viên khác có thể viết:

```
sinhVienA.diemTrungBinh = -9.5;// Vô lý!
sinhVienA.maSV = null // Mất dữ liệu!
```

Hậu quả: Dữ liệu bị thay đổi bừa bãi, không kiểm soát → trạng thái không hợp lệ → lỗi nghiêm trọng!



Thiếu kiểm soát truy cập như để cửa mở không khóa

Giải pháp: TÍNH ĐÓNG GÓI (ENCAPSULATION)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Bổ từ truy cập (Access modifiers)

Rikkei Academy
where the dream begins

Tính đóng gói

Định Nghĩa

Tính đóng gói là một trong 4 trụ cột của OOP. Nó **che giấu (hiding)** dữ liệu và chi tiết cài đặt bên trong đối tượng, chỉ cung cấp **giao diện công khai (public interface)** để tương tác có kiểm soát.

Mục đích của Tính đóng gói:

Bảo vệ dữ liệu: Ngăn chặn truy cập trái phép

Giảm phức tạp: Chỉ cần biết cách sử dụng, không cần biết cài đặt

Tăng bảo mật: Kiểm soát được ai, cái gì có thể thay đổi dữ liệu

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Bổ từ truy cập (Access modifiers)

Access modifiers

+ Public

"Không khóa"

Ai cũng có thể truy cập từ bất cứ đâu. Hoàn toàn công khai và minh bạch.

Thường dùng cho: Phương thức công khai

- Private

"Khóa riêng tư"

Chỉ có thể truy cập từ BÊN TRONG chính Class đó. Hoàn toàn bảo mật.

Thường dùng cho: Thuộc tính nhạy cảm

Protected

"Khóa gia đình"

Truy cập được từ Class và các Class con kế thừa. Chia sẻ trong gia đình.

Thường dùng cho: Dữ liệu kế thừa

~ Package

"Khóa hàng xóm"

Các Class trong cùng package có thể truy cập. Chia sẻ trong cộng đồng.

Thường dùng cho: Dữ liệu nội bộ

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Bổ từ truy cập (Access modifiers)

Tương tác với dữ liệu private

→ GETTER & SETTER

1

GETTER

Phương thức public dùng để **LẤY (GET)** giá trị của thuộc tính private

```
getHoTen(): String
getDiemTrungBinh(): float
```

Như "cửa sổ xem" - chỉ đọc, không thay đổi

2

SETTER

Phương thức public dùng để **THIẾT LẬP (SET)** giá trị cho thuộc tính private

```
setHoTen(String hoTen): void
setDiemTrungBinh(float diem): void
```

Như "cửa có bảo vệ" - kiểm tra trước khi cho vào

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Bổ từ truy cập (Access modifiers)

Protected (#) và Package (~)

Protected - "Khóa Gia Đình"

Mức độ: Mạnh hơn private, yếu hơn public

Ai truy cập được: Bản thân Class + các Class con kế thừa

Khi nào dùng: Muốn che giấu với thế giới bên ngoài nhưng vẫn để lớp con kế thừa truy cập trực tiếp

(Sẽ rõ hơn khi học về Inheritance)

~ Package - "Khóa Hàng Xóm"

Mức độ: Mạnh hơn private, yếu hơn protected

Ai truy cập được: Các Class trong cùng package

Khi nào dùng: Các class có liên quan chặt chẽ, được gom chung vào một "gói" (package), cần truy cập lẫn nhau

Như các gia đình trong cùng khu phố

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

3. Các mối quan hệ

Phân loại các mối quan hệ

> "Một Class Diagram không chỉ là các hình chữ nhật, mà là cách chúng được kết nối."

Để dễ hình dung, chúng ta sẽ chia các mối quan hệ thành 3 nhóm lớn dựa trên bản chất của chúng:



Quan hệ Cấu trúc



Quan hệ Phân loại



Quan hệ Phụ thuộc

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

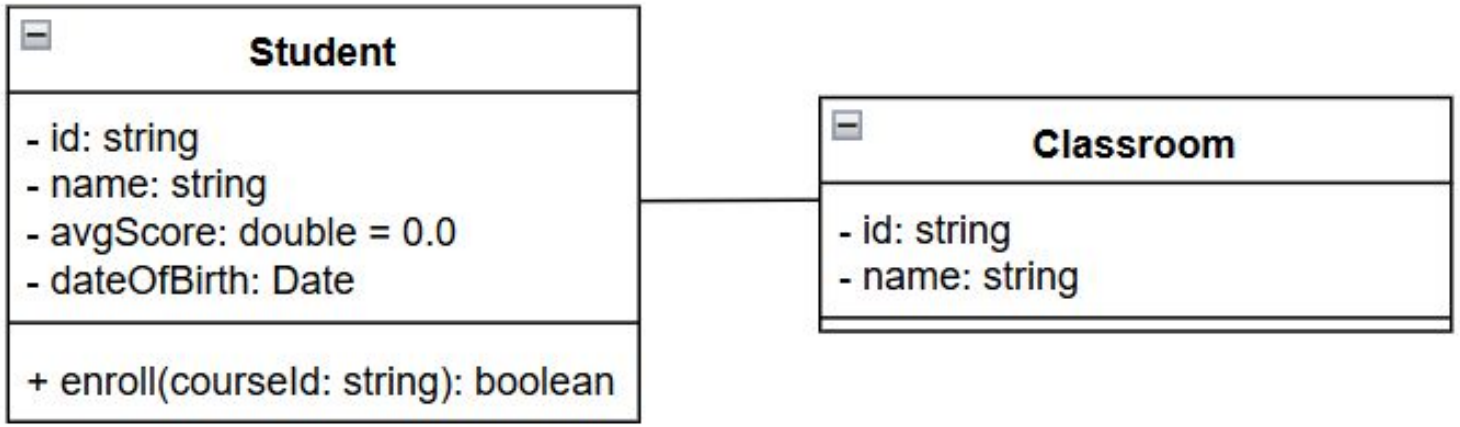
3. Các mối quan hệ

Quan hệ cấu trúc: Association

Association (hai chiều)

Ký hiệu: _____ (Đường thẳng)

Ý nghĩa: Mối liên kết cấu trúc, lâu dài. Cả hai đối tượng đều "biết" về nhau.

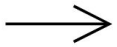


© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

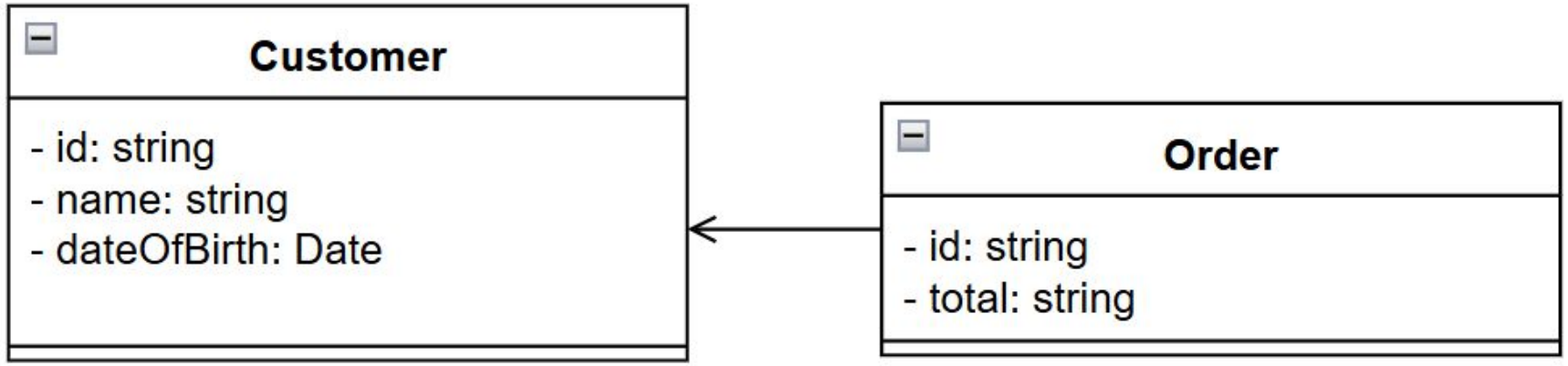
3. Các mối quan hệ

Quan hệ cấu trúc: Association

Directed Association (một chiều)

Ký hiệu:  (Đường thẳng có mũi tên)

Ý nghĩa: Chiều mũi tên chỉ ra hướng "nhận biết". Class nguồn biết về Class đích, nhưng Class đích không cần biết về Class nguồn.

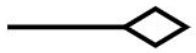


© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

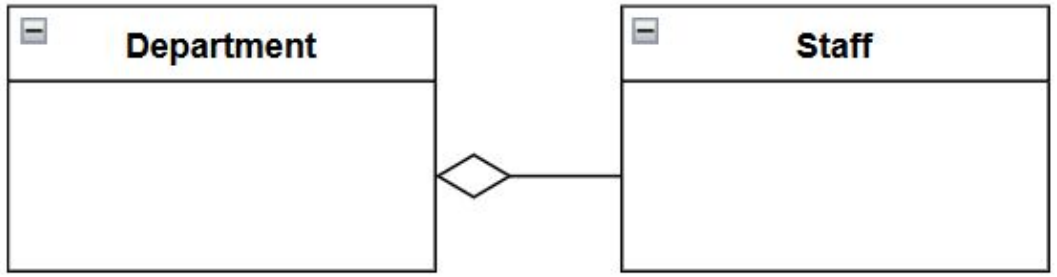
3. Các mối quan hệ

Quan hệ cấu trúc: Aggregation & Composition

Aggregation (chứa đựng)

Ký hiệu:  (Hình thoi rỗng ở phía Toàn thể)

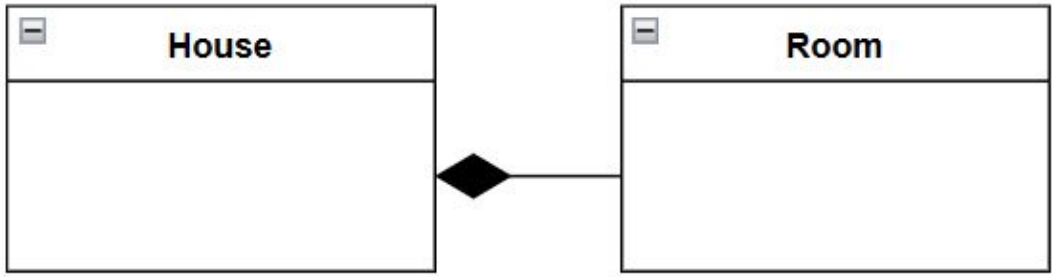
Ý nghĩa: Quan hệ lỏng lẻo. Vòng đời của Bộ phận **độc lập** với Toàn thể.



Composition (sở hữu/cấu thành)

Ký hiệu:  (Hình thoi đặc ở phía Toàn thể)

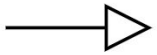
Ý nghĩa: Quan hệ chặt chẽ. Vòng đời của Bộ phận **phụ thuộc hoàn toàn** vào Toàn thể.



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

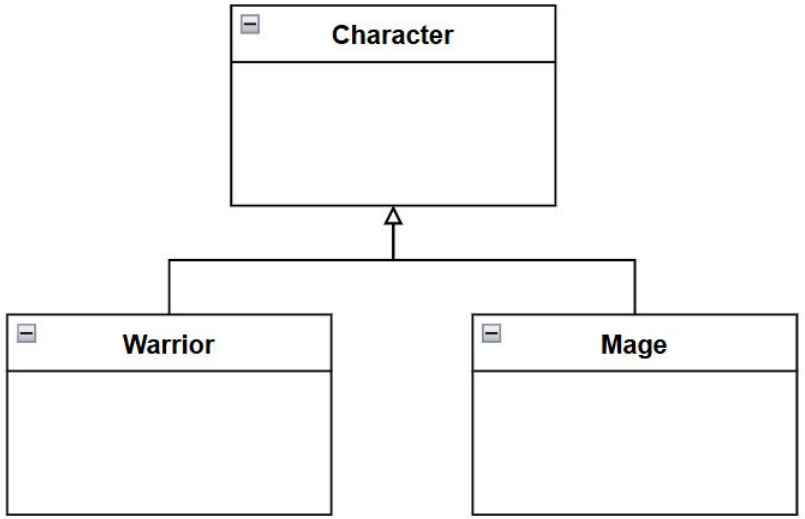
3. Các mối quan hệ

Quan hệ phân loại: Generalization

Ký hiệu:  (Mũi tên nét liền, tam giác rỗng, chỉ từ Con lên Cha)

> **Ý nghĩa:** Mối quan hệ "is-a-kind-of" (là một loại của...). Class Con (Subclass) **kế thừa** các thuộc tính và phương thức từ Class Cha (Superclass).

Đây là mối quan hệ giữa các **Class**, thể hiện tính kế thừa trong lập trình hướng đối tượng.



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

3. Các mối quan hệ

Quan hệ phân loại: Realization

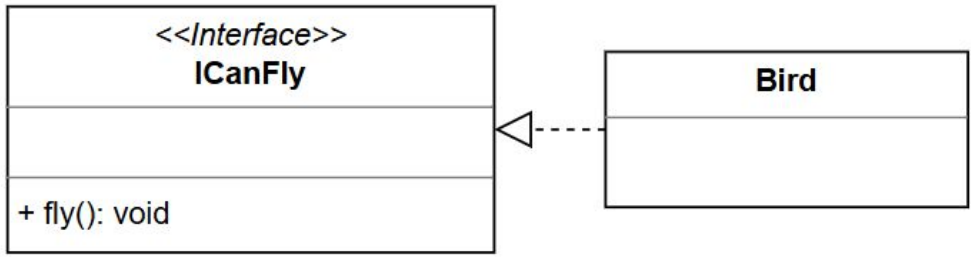
Ký hiệu: ----▷ (Mũi tên nét **đứt**, tam giác rỗng, chỉ từ Class đến Interface)

Ý nghĩa

Mối quan hệ "can-do" (có khả năng làm...). Một Class **cam kết** sẽ cung cấp mã nguồn (hiện thực hóa) cho các "hợp đồng" (phương thức) được định nghĩa trong một **Interface**.

Đặc điểm

Đây là mối quan hệ giữa một **Class và một Interface**. Class phải implement tất cả các phương thức được khai báo trong Interface.



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

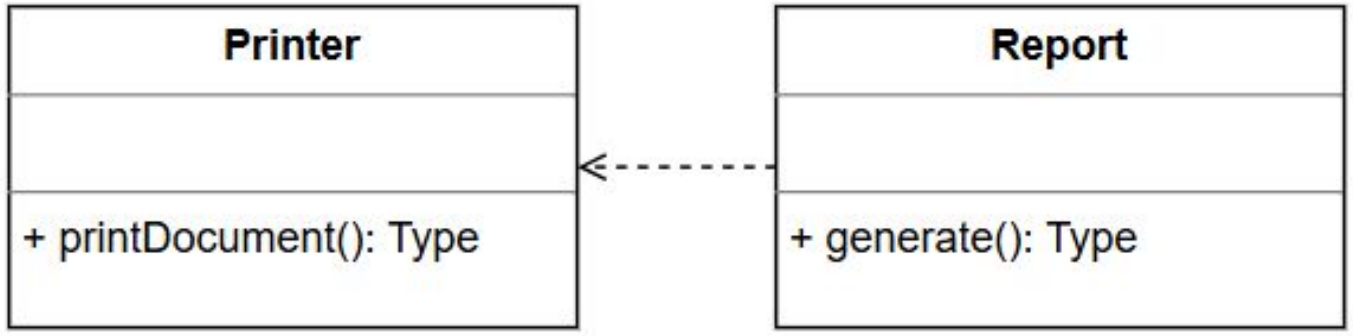
3. Các mối quan hệ

Quan hệ phụ thuộc: Dependency

Dependency

Ký hiệu: ------> (Mũi tên nét đứt)

Ý nghĩa: Một thay đổi trong Class B (Supplier) có thể ảnh hưởng đến Class A (Client). Class A phụ thuộc vào định nghĩa của Class B.



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

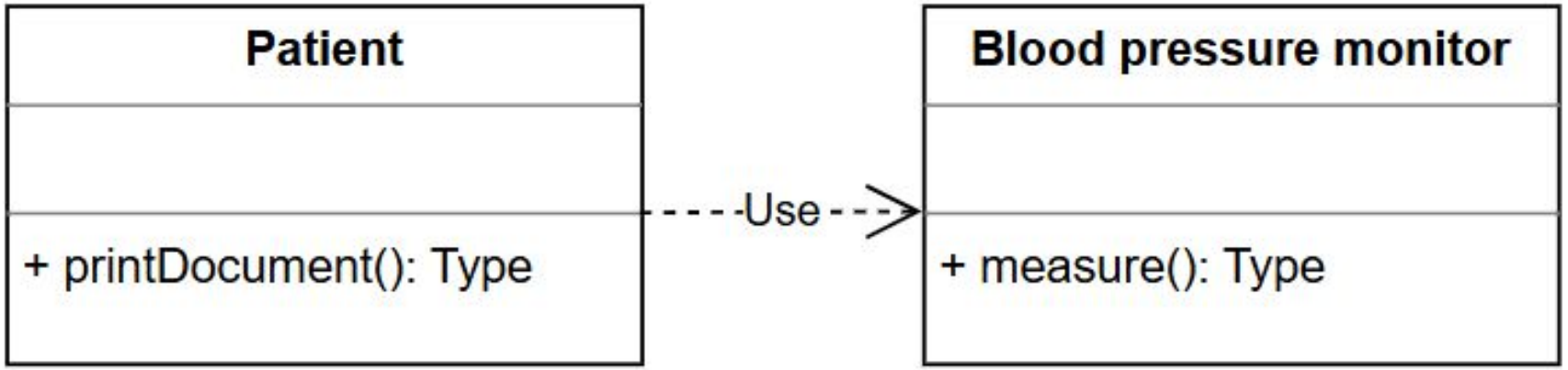
3. Các mối quan hệ

Quan hệ phụ thuộc: Usage

Usage

Ký hiệu: ----Use--> Là một trường hợp cụ thể, rõ ràng hơn của Dependency, thường được gắn nhãn `<<use>>`.

Ý nghĩa: Class A sử dụng một đối tượng của Class B một cách tạm thời (làm tham số phương thức, biến cục bộ).



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Bội số trong Class Diagram

Câu hỏi quan trọng

Chúng ta đã biết cách nối các Class bằng các đường quan hệ (Association, Aggregation...). Ví dụ: SinhVien --- LopHoc.

Vấn đề

Đường nối này chỉ cho biết **"có"** mối quan hệ, nhưng không cho biết **số lượng**.

Câu hỏi nghiệp vụ

Một LopHoc có thể có **bao nhiêu** SinhVien? (Tối thiểu? Tối đa?)

Một SinhVien có thể đăng ký **bao nhiêu** LopHoc?

> Để trả lời câu hỏi "Bao nhiêu?", chúng ta cần một công cụ gọi là **Multiplicity** (Bội số).



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Bội số trong Class Diagram

Multiplicity là gì?

> **Định nghĩa:** Multiplicity là một **ràng buộc (constraint)** được đặt trên một mối quan hệ, chỉ rõ **số lượng đối tượng** của một Class có thể liên kết với **MỘT** đối tượng của Class kia.

01

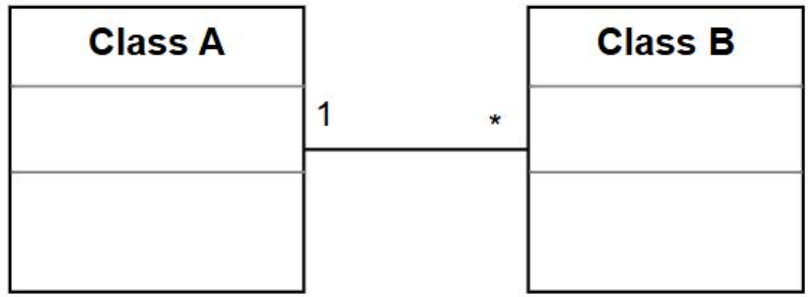
Vị trí đặt

Nó được đặt ở mỗi đầu của một đường quan hệ.

02

Cách đọc

Khi xem Multiplicity ở đầu Class B, hãy đọc là: "**MỘT** đối tượng của Class A sẽ liên kết với **[Multiplicity]** đối tượng của Class B."



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Bội số trong Class Diagram

Các ký hiệu phổ biến

Ký hiệu	Ý nghĩa	Ví dụ đọc
1	Đúng một (Exactly one)	Một Hóa đơn phải thuộc về **đúng một** Khách hàng
*****	Không hoặc nhiều (0 or more)	Một Khách hàng có thể có **không hoặc nhiều** Hóa đơn
0..1	Không hoặc một (0 or 1)	Một Nhân viên có thể quản lý **không hoặc một** Phòng ban
1..*	Một hoặc nhiều (1 or more)	Một Lớp học phải có **một hoặc nhiều** Sinh viên
m..n	Từ m đến n (Specific range)	Một Lớp học có từ **25 đến 40** Sinh viên

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Bội số trong Class Diagram

Quan hệ 1 - 1

1

Ví dụ 1: Người và CMND

Từ CMND: Một Nguoi có đúng một ChungMinhNhanDan

Từ Nguoi: Một ChungMinhNhanDan thuộc về đúng một Nguoi

2

Ví dụ 2: Nhân viên và Máy tính

Từ Máy tính: Một NhanVien có thể có không hoặc một MayTinhCongTy

Từ Nhân viên: Một MayTinhCongTy phải thuộc về đúng một NhanVien

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Bội số trong Class Diagram

Quan hệ 1 - n

Ví dụ 1: Khách hàng và Đơn hàng

Phân tích & Đọc:

Từ Đơn hàng: Một KhachHang có thể có không hoặc nhiều DonHang

Từ Khách hàng: Một DonHang phải thuộc về đúng một KhachHang

Ví dụ 2: Lớp học và Sinh viên

Phân tích & Đọc:

Từ Lớp học: Một LopHoc phải có **một hoặc nhiều** SinhVien

Từ Sinh viên: Một SinhVien thuộc về **đúng một** LopHoc

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Bội số trong Class Diagram

Quan hệ n - n

Ví dụ: Sinh viên và Môn học

Phân tích & Đọc:

Từ Môn học: Một SinhVien có thể đăng ký **không hoặc nhiều** (*) MonHoc.

Từ Sinh viên: Một MonHoc có thể được đăng ký bởi **không hoặc nhiều** (*) SinhVien.

Để triển khai quan hệ N-N trong cơ sở dữ liệu, chúng ta cần một **bảng trung gian** (junction table) để phá vỡ nó thành hai quan hệ 1-N. Bảng trung gian này sẽ chứa khóa chính của cả hai thực thể liên quan.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Bội số trong Class Diagram

Tầm quan trọng của multiplicity

Làm rõ quy tắc nghiệp vụ

Biến những quy tắc mơ hồ ("một lớp có sinh viên") thành những ràng buộc chính xác ("một lớp phải có **TỐI THIỂU 1** và **TỐI ĐA 40** sinh viên").

Đầu vào cho thiết kế CSDL

Quan hệ 1-N dẫn đến việc tạo **khóa ngoại**. Quan hệ N-N bắt buộc phải tạo **bảng trung gian**.

Hỗ trợ lập trình chính xác

Giúp lập trình viên xác định kiểu dữ liệu của thuộc tính (một đối tượng hay danh sách đối tượng) và các logic kiểm tra cần thiết.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

5. Chuyển đổi Use Case sang Class Diagram

Kết nối giữa use case và diagram

Thế giới 1: Phân tích Hành vi

Công cụ: Use Case Diagram, Mô tả Use Case

Câu hỏi: Hệ thống **LÀM GÌ**?

- Xác định các chức năng
- Mô tả tương tác với người dùng
- Định nghĩa luồng hoạt động

Thế giới 2: Phân tích Cấu trúc

Công cụ: Class Diagram

Câu hỏi: Hệ thống **GỒM NHỮNG GÌ**?

- Xác định các lớp đối tượng
- Định nghĩa thuộc tính và phương thức
- Thiết lập mối quan hệ

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

5. Chuyển đổi Use Case sang Class Diagram

Rikkei Academy® where the dream begins

Tầm quan trọng của multiplicity

KỸ THUẬT: PHÂN TÍCH DANH TỪ - ĐỘNG TỪ

Đây là một kỹ thuật đơn giản nhưng rất mạnh mẽ để "quét" qua các tài liệu yêu cầu, đặc biệt là bản Mô tả chi tiết Use Case.

DANH TỪ (Nouns)

Tìm tất cả các **DANH TỪ** hoặc cụm danh từ

→ Chúng là những **ỨNG CỬ VIÊN** cho **CLASSES** hoặc **ATTRIBUTES**

ĐỘNG TỪ (Verbs)

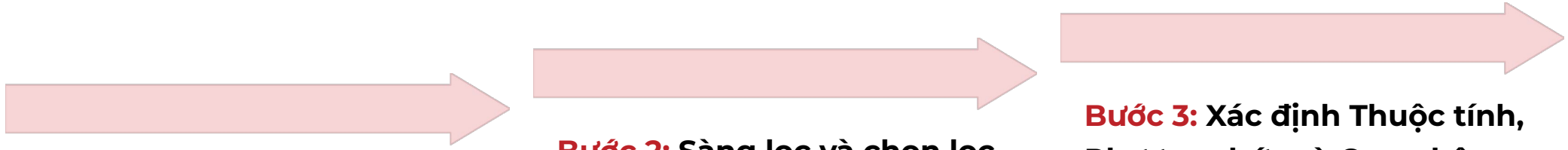
Tìm tất cả các **ĐỘNG TỪ** hoặc cụm động từ

→ Chúng là những **ỨNG CỬ VIÊN** cho **METHODS** hoặc **RELATIONSHIPS**

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

5. Chuyển đổi Use Case sang Class Diagram

Quy trình 3 bước xây dựng class diagram sơ bộ



Bước 1: Tìm các ứng viên Class

Bước 2: Sàng lọc và chọn lọc các Class

Bước 3: Xác định Thuộc tính, Phương thức và Quan hệ

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

5. Chuyển đổi Use Case sang Class Diagram

case study: Phân tích use case đăng ký môn học

Bước 1: Tìm ứng viên Class

> Một Sinh viên đăng nhập vào Hệ thống để xem danh sách các Lớp học phần đang mở. Mỗi Lớp học phần thuộc về một Môn học cụ thể. Môn học có mã, tên, và số tín chỉ. Sinh viên chọn một Lớp học phần để thực hiện đăng ký. Hệ thống sẽ kiểm tra điều kiện tiên quyết và lịch học của sinh viên để tránh trùng lặp. Nếu thành công, một phiếu đăng ký sẽ được tạo ra và lưu lại kết quả.

Danh sách ứng viên đã tìm được:

Sinh viên, Hệ thống, danh sách, Lớp học phần, Môn học, mã, tên, số tín chỉ, đăng ký, điều kiện tiên quyết, lịch học, phiếu đăng ký, kết quả

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

5. Chuyển đổi Use Case sang Class Diagram

Bước 2: sàng lọc và chọn lọc các class

Ứng viên	Quyết định	Lý do
Sinh viên	Giữ lại	Là một thực thể quan trọng, có thông tin và hành vi riêng
Lớp học phần	Giữ lại	Thực thể chính trong nghiệp vụ đăng ký
Môn học	Giữ lại	Thực thể chính, chứa thông tin về môn học
Phiếu đăng ký	Giữ lại	Class để lưu thông tin về một lần đăng ký
Hệ thống	Loại bỏ	Quá chung chung, đại diện cho toàn bộ hệ thống
danh sách	Loại bỏ	Cấu trúc dữ liệu, không phải thực thể nghiệp vụ
mã, tên, số tín chỉ	→ Thuộc tính	Là đặc điểm của Môn học

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

5. Chuyển đổi Use Case sang Class Diagram

Bước 3: Lắp ghép các thành phần

01

Xác định Thuộc tính (Attributes)

02

Xác định Phương thức (Methods)

03

Thiết lập Mối quan hệ (Relationships)

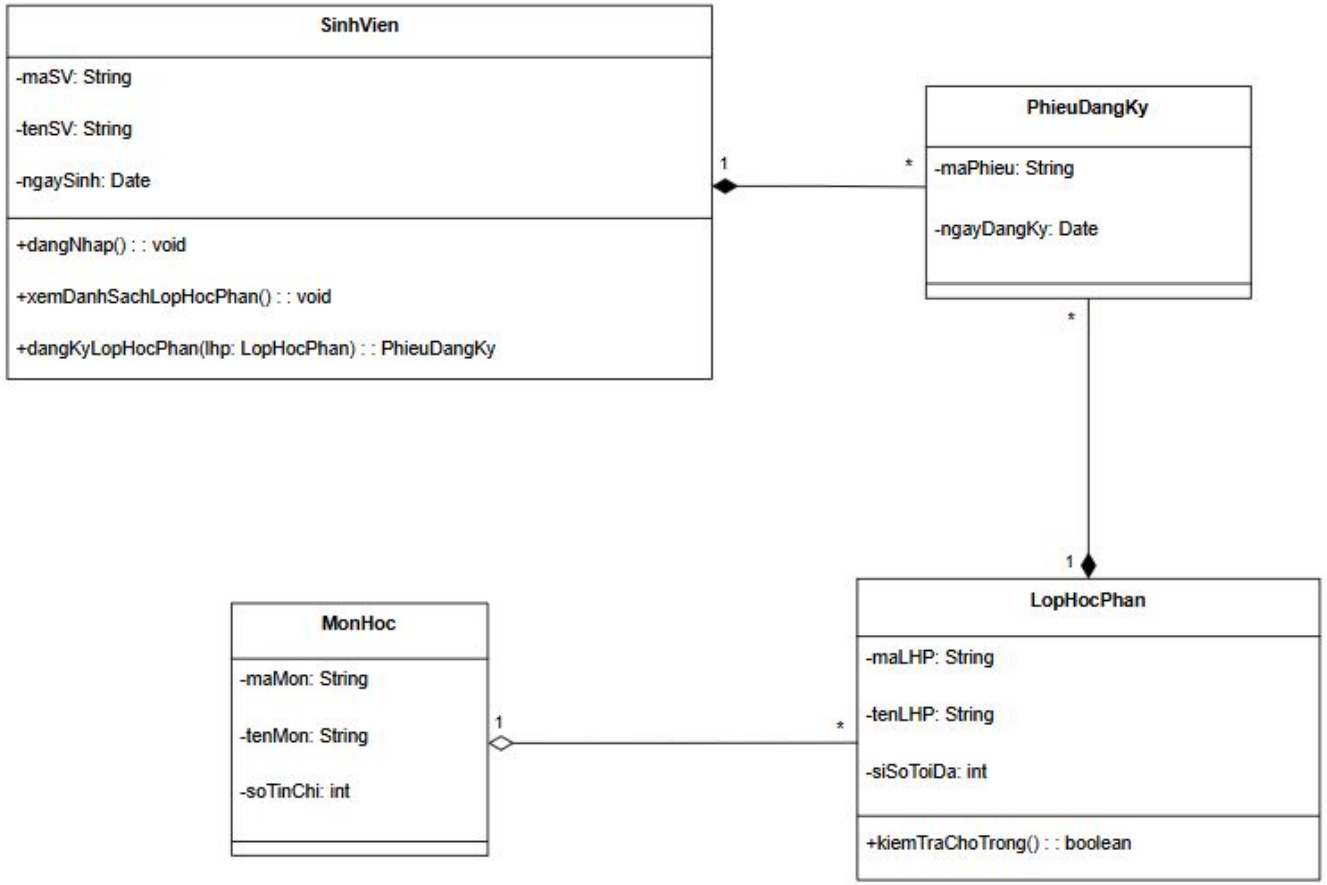
04

Hoàn thiện Class Diagram

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

5. Chuyển đổi Use Case sang Class Diagram

Kết quả class diagram sơ bộ



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN